ECEN 248 - Lab Report

Lab #7

Introduction to Sequential Logic

Section: 524

Ramya Kannan

UIN: 635002011

11/6/2025

TA: Reed Clardy

Objectives:

The objective of this is to introduce sequential logic circuits through data components such as latches and flip-flops. In this lab, we will further enhance our knowledge of Verilog coding. We will describe these data components at the gate-level using structural and behavioral Verilog. We will explore synchronous sequential circuits by combining flip-flops with combinational logic, as we have done in previous labs. This lab also introduces simulation delays into combinational logic and observes the effects on clock timing. The goal of this lab is to gain skills in designing sequential circuits, understanding the differences between latches and flip-flops, and learning how propagation delays impact synchronous circuit operation.

Design:

This lab implemented sequential logic design using structural and behavioral modeling in the Xilinx Vivado Design Suite. The design process began with simulating fundamental memory elements and advanced to constructing a synchronous adder circuit.

We first developed an SR-latch using structural Verilog with gate-level primitives. The sr_latch module featured outputs Q and notQ declared as wires, and inputs En, S, and R. Four NAND gates were instantiated with 2ns delays to simulate gate propagation. The intermediate nets nandSEN and nandREN linked the input NAND gates to the cross-coupled output NAND gates. We verified the module with sr_latch_tb.v. Subsequently, we adjusted all delays from 2ns to 4ns to examine how increased propagation delay affected circuit timing through waveform analysis.

Next, we built a D-latch using structural Verilog. The d_latch module contained one NOT gate and four NAND gates, each with 2ns delays. The enable signal En determined whether output Q tracked input D when En = 1 or retained its previous value when En = 0. We tested and verified this module with d_latch_tb.v.

Then, we utilized two D-latches in a master-servant configuration to create a positive-edge-triggered D flip-flop. The d_flip_flop module instantiated two d_latch modules and two NOT gates with 2ns delays. When the clock (Clk) transitioned from low to high, the master latch captured input D while the servant latch became transparent, transferring the master's output to the flip-flop output Q. We verified this module with d_flip_flop_tb.v and analyzed internal signals Qm and notClk.

We then created the d_latch_behavioral module to implement behavioral descriptions of memory components. This module used an always block triggered by changes in En or D. The output Q was declared as type reg since it was assigned within an always block. When En was high, Q received the value of D through blocking assignment. We verified this module using its corresponding testbench file. Subsequently, we developed the d_flip_flop_behavioral module. This module used an always@(posedge Clk) block to describe edge-sensitive behavior. The output Q was assigned D using non-blocking assignment (<=) to ensure correct sequential operation. We verified this module with its corresponding testbench file.

In the second part of the experiment, we constructed a 2-bit synchronous adder by integrating combinational and sequential logic. We retrieved the full_adder module from a previous lab (Lab 5) and incorporated delays into the assigned statements: 6ns for 3-input gates and 4ns for 2-input gates. Using two instances of full_adder, we assembled the adder_2bit module with a 2-bit sum output and carry output. We then completed all 16 test cases for the adder_2bit testbench and determined the worst-case propagation delay through the ripple-carry adder via simulation waveform analysis. Finally, we implemented the complete synchronous adder circuit. The adder_synchronous module instantiated the adder_2bit module and specified three register stages using behavioral always@(posedge Clk) blocks. Two 2-bit registers (A_reg and B_reg) held the input operands, while a 2-bit register (Sum_reg) and a 1-bit register (Carry_reg) stored the computation results. All registers utilized non-blocking assignments to maintain proper synchronous operation. We tested this circuit with its corresponding testbench file and experimented with various clock periods until all tests passed, beginning from CLOCK_PERIOD = 18ns.

Results:

This lab successfully verified and simulated the behavioral and structural Verilog modules in Xilinx Vivado Design Suite. All sequential logic components were tested using their corresponding testbenches and analyzed through waveforms.

Figure 9 shows the SR-latch simulation with 2ns delays passing all test cases. The circuit executed set, reset, and hold operations correctly. When En = 0, Q and notQ preserved their current values independent of S and R. When En = 1 with S=1 and R=0, Q became 1, and with S=0 and R=1, Q became 0. Modifying the delays to 4ns in Figure 10 resulted in failed tests for reset, set, and reset from set operations because the longer propagation time violated timing requirements.

The D-latch waveform in Figure 11 exhibited proper level-sensitive characteristics. When En = 1, Q mirrored D after the propagation delay. When En = 0, Q stored its existing value, demonstrating the storage capability. The behavioral implementation of D-latch, as seen in Figure 12, replicated the functionality but made immediate transitions, since behavioral modeling using always blocks lacked timing delays, while the structural design displayed actual gate-level propagation.

The D flip-flop in Figure 13 showed correct edge-triggered functionality through its master-servant design. During low Clk, Qm tracked D while Q held constant. When Clk transitioned high, the master sampled D and the servant output Qm to Q. Post-edge D modifications did not affect Q until the next rising transition. Internal signals Qm and notClk validated the master-servant configuration. The behavioral implementation of D Flip-flop, as seen in Figure 14, functioned identically to the structural behavior, and had immediate transitions without propagation delays.

The 2-bit ripple-carry adder in Figure 15 produced accurate results for all 16 input scenarios. Observing the carry path through both adder stages showed a 20ns maximum delay. The synchronous adder testing revealed clock-dependent performance. Figure 16 at 18ns showed multiple incorrect Sum_reg and Carry_reg values. Progressively lengthening the clock period demonstrated successful operation at 20ns in Figure 18, identifying the minimum period required for dependable synchronous functionality.

Conclusion:

In this lab, we successfully introduced sequential logic circuits through the implementation and simulations of latches, flip-flops, and 2-bit adders. All modules were successfully tested and verified in Vivado. We enhanced our knowledge in Verilog coding by describing data components at both the gate-level using structural and behavioral Verilog (always blocks). Through the SR-latch, D-latch, D flip-flop, and 2-bit adder, we furthered our understanding of the differences between level-sensitive latches, edge-triggered flip-flops, and how propagation delays affect synchronous operation. The synchronous adder demonstrated that clock timing must account for worst-case propagation paths, as periods below 20ns caused combinational logic to fail settling before subsequent clock edges. Overall, this lab provided valuable experience in designing sequential circuits and understanding timing constraints.

Post-Lab Deliverables:

1. **For 1(f) in part 1, explain the results of the simulation**
   Changing it from 2ns to 4ns resulted in a failure of tests. With a 2ns delay, the simulation successfully passed all the tests as seen in the console output. This means that the SR-latch demonstrated the changes in input. In the 4ns delay, several tests failed, including the SR-latch reset test, set test, and reset from set test. This happened because when we increased the delay time, that also increased the total propagation time, so that means the outputs were much more delayed than we expected.

2. **For 3 in part 1, check the waveform with the internal signals. Are the latches behaving as expected? Why or why not?**
   Yes, the latches behave correctly for the master-servant configuration. In Figure 13, Qm tracks input D while Clk is low, and Q holds its previous value. On Clk's rising edge, the master latch freezes Qm and the servant latch updates Q to match Qm. The notClk signal confirms the master and servant receive complementary enable signals. This creates edge-triggered operation where output Q only changes at rising clock edges, not throughout the high clock level, which is exactly the expected behavior for a properly functioning master-servant D flip-flop.

3. **For 1.4 in part 1, compare the waveforms you captured from the behavioral Verilog to those from the structural Verilog. Are they different? If so, how?**
   Examining the waveforms for structural vs behavioral for both D-latch and D flip-flop, the logical functionality remains equivalent. Both of the versions generate the correct output patterns. However, structural Verilog waveforms demonstrate propagation delays since gate-level primitives incorporate explicit delays (2ns per NAND gate and inverter). The behavioral Verilog waveforms perform immediate transitions since the always blocks execute without simulated gate delays. The behavioral models show functional accuracy rather than timing precision, which is more applicable for synthesis purposes.

4. **For 1(e) in part 2, what is the worst-case propagation delay through the adder?**

The worst-case propagation delay was determined by analyzing the 2-bit adder simulation waveform, specifically examining the Carry signal's settling time after input transitions. The longest delay occurred when input values changed from combinations producing no carry to combinations requiring carry generation, forcing sequential propagation through both full adder stages. Based on the specified gate delays of 6ns for 3-input gates and 4ns for 2-input gates, along with waveform measurements, the worst-case propagation delay was 20ns. This measurement is validated by the synchronous adder testing results: the circuit failed at clock periods of 18ns and 19ns because these periods were insufficient for the combinational logic to settle, but succeeded when the clock period reached 20ns.

5. **Based on the clock period you measured for your synchronous adder, what would be the theoretical maximum clock rate (frequency)? What would be the effect of increasing the width of the adder on the clock rate? How might you improve the clock rate of the design?**
   It was found that the minimum clock period of 20ns was necessary for complete test passage; the maximum clock rate is f = 1/T, which is 1/20ns = 50MHz.
   If we were to increase the adder-bit width, it would reduce the clock rate because the design of this ripple-carry has a propagation delay that grows linearly with the number of bits. The worst-case propagational delay would grow because the carry must ripple through all stages as well as the new ones. One way that the clock rate could be improved is to use a carry-lookahead adder instead, because this will reduce the delay.

<u>Student Feedback</u>

1. **What did you like most about the lab assignment and why? What did you like least about it and why?**
   I liked creating the testbenches.
2. **Were there any sections of the lab manual that were unclear? If so, what was unclear? Do you have any suggestions for improving the clarity?**
   No, everything in the manual was clear.
3. **What suggestions do you have to improve the overall lab assignment?**
   No, I do not have suggestions for this lab.

Figure 1. 2ns

```
1  `timescale 1ns / 1ps
2
3  module sr_latch(Q, notQ, En, S, R);
4      //PORTLIST
5      output wire Q, notQ;
6      input wire En, S, R;
7
8      //INTERMEDIATE  NETS
9
10     wire nandSEN, nandREN;
11
12     //delay gate nand0 is 2ns delay
13     nand #2 nand0(Q, nandSEN, notQ);
14     nand #2 nand1(notQ, nandREN, Q);
15     nand #2 nand2(nandSEN, S, En);
16     nand #2 nand3(nandREN, R, En);
17 endmodule // sr_latch
```

Figure 2. 4 ns

```
`timescale 1ns / 1ps

module sr_latch(Q, notQ, En, S, R);
    //PORTLIST
    output wire Q, notQ;
    input wire En, S, R;

    //INTERMEDIATE  NETS

    wire nandSEN, nandREN;

    //delay gate nand0 is 2ns delay
    nand #4 nand0(Q, nandSEN, notQ);
    nand #4 nand1(notQ, nandREN, Q);
    nand #4 nand2(nandSEN, S, En);
    nand #4 nand3(nandREN, R, En);
endmodule // sr_latch
```

Figure 3. D-Latch Structural

```
`timescale 1ns / 1ps

module d_latch(Q, notQ, En, D);
  // delcaring the wires
   output wire Q; //primary output
   output wire notQ; //compliemnt output
   input wire En; //enable signal
   input wire D;

   // now the intermediate wires
   wire Dnot;
   wire nandDnotEN;
   wire Dn1;

   //structural for d latch
    not #2 (Dnot, D); //created complement of D, 2ns
    nand #2 nand0(nandDEN, D, En); //when EN = 1 and D = 1, output witll be 0
    nand #2 nand1(Q, nandDEN, notQ);
    nand #2 nand2 (nandDnotEN, Dnot, En);
    nand #2 nand3 (notQ, Q, nandDnotEN);

    //When En=0: Both nandDEN and nandDnotEN are 1, latch holds previous state
    // When En=1: D value propagates through - if D=1, Q=1; if D=0, Q=0
endmodule
```

Figure 4. D Flip-Flop Structural

```
`timescale 1ns / 1ps

module d_flip_flop (Q, notQ, Clk, D);
//ports
  output wire Q, notQ;
  input wire Clk, D;

  // internal nets
  wire notClk, notNotClk;
  wire Q_m; // output of the master latch
  // notQ_m will be wired to the d_latch but then left
  wire notQ_m;


  // instantiate and wire up the not gates here
  not #2 not0(notClk, Clk);
  not #2 not1(notNotClk, notClk);

  // instantiate and wire up the d latches
  d_latch dlMaster(Q_m, notQ_m, notClk, D);
  d_latch dlServant(Q, notQ, notNotClk, Q_m);
endmodule // d_flip_flop
```

Figure 5. D-Latch Behavioral

```
`timescale 1ns / 1ps

module d_latch_behavioral(output reg Q, output wire notQ, input wire D, En);

    //descibr behavior of d-latch
    always@(En or D)
    if(En)
        Q=D;
    else
        Q=Q; //this is implied but i want to add it
    assign notQ = ~Q; //reg can drive wires
endmodule
```

Figure 6. D Flip-Flop Behavioral

```
`timescale 1ns / 1ps

module d_flip_flop_behavioral(output reg Q, output wire notQ, input wire D, input wire Clk);//clk clock siganl

    always@(posedge Clk) //trigger when Clk goes High
        Q <= D; //non blocking assigment statement
    assign notQ = ~Q;
endmodule
```

Figure 6. 2-bit Adder

```
`timescale 1ns / 1ps

module adder_2bit(Carry, Sum, A, B);
    output wire Carry;
    output wire [1:0] Sum; //1 bit
    input wire [1:0] A, B; //1 always
    wire c0; //intialiaing the internal wire
    //creaing 2 full adders
    full_adder FA0(Sum[0], c0, A[0], B[0], 1'b0);

    full_adder FA1(Sum[1], Carry, A[1], B[1], c0);

endmodule
```

Figure 7. Filled out Testbench for 2-bit Adder

```
`timescale 1ns / 1ps

module adder_2bit_tb;//a test bench does not have any ports of its own!

    /* Input nets */
    reg [1:0] A; //these are regs because they are modified in
    reg [1:0] B; //a behavioral block

    /* Output nets */
    wire [1:0] Sum; //these are wires because they will be driven
    wire Carry;//by the inantiated module

    /* Instantiate the Unit Under Test (UUT) */
    adder_2bit uut ( //this is a different way
        .A(A),       //to instantiate a module.
        .B(B),       //the nice thing about this style
        .Sum(Sum),   //is that the order does not matter!
        .Carry(Carry)//notice the ports are in a different order!
    );


    /*-this is a behavioral block which is executed only once!  *
     *-the statements within this behavioral block are executed *
     *-sequentially because we are using blocking statements    *
     *-an '=' sign within a behavioral construct is considered a*
     * blocking statement. We will talk more about this later...*/
    initial
      begin

        /* Initialize inputs*/
        A = 0;
        B = 0;

        #25; //just delay 25 ns
        {A,B} = 4'b0000; //stimulate the inputs
        #25; //wait a bit for the result to propagate
        //here is where we could put a check to see if the results
        //are as expected!
        if({Carry, Sum} != 3'b000)//you could put your own message here
            $display("Ah crap... something went wrong here...");
            else
                $display("Hey! The UUT passed this test vector...");
        //let's do it again with a different input...
        {A,B} = 4'b0001; //stimulate the inputs
        #25; //wait a bit for the result to propagate
        //check output
        if({Carry, Sum} != 3'b001)
            $display("Your own message here...");
            else
                $display("Test vector passed!!!");
        //okay this is fun... you try it now...

        //go through all possible input combinations (2^4 = 16)
        //cut and paste makes this task a lot easier
```

```
        {A,B} = 4'b0010; //stimulate the inputs
        #25; //wait a bit for the result to propagate
        //check output
        if({Carry, Sum} != 3'b010)
            $display("Your own message here...");
            else
                $display("Test vector passed!!!");
        //okay this is fun... you try it now...

        //go through all possible input combinations (2^4 = 16)
        //cut and paste makes this task a lot easier

        //when we are done, let's stop the simulation

        {A,B} = 4'b0011; //stimulate the inputs
        #25; //wait a bit for the result to propagate
        //check output
        if({Carry, Sum} != 3'b011)
            $display("Your own message here...");
            else
                $display("Test vector passed!!!");
        //okay this is fun... you try it now...

        //go through all possible input combinations (2^4 = 16)
        //cut and paste makes this task a lot easier

        //when we are done, let's stop the simulation

        {A,B} = 4'b0100; //stimulate the inputs
        #25; //wait a bit for the result to propagate
        //check output
        if({Carry, Sum} != 3'b001)
            $display("Your own message here...");
            else
                $display("Test vector passed!!!");
        //okay this is fun... you try it now...

        //go through all possible input combinations (2^4 = 16)
        //cut and paste makes this task a lot easier

        //when we are done, let's stop the simulation

        {A,B} = 4'b0101; //stimulate the inputs
        #25; //wait a bit for the result to propagate
        //check output
        if({Carry, Sum} != 3'b010)
            $display("Your own message here...");
            else
                $display("Test vector passed!!!");
        //okay this is fun... you try it now...

        //go through all possible input combinations (2^4 = 16)
        //cut and paste makes this task a lot easier

        //when we are done, let's stop the simulation
```

```
{A,B} = 4'b0110; //stimulate the inputs
#25; //wait a bit for the result to propagate
//check output
if({Carry, Sum} != 3'b011)
    $display("Your own message here...");
    else
        $display("Test vector passed!!!");
//okay this is fun... you try it now...

//go through all possible input combinations (2^4 = 16)
//cut and paste makes this task a lot easier

//when we are done, let's stop the simulation

{A,B} = 4'b0111; //stimulate the inputs
#25; //wait a bit for the result to propagate
//check output
if({Carry, Sum} != 3'b100)
    $display("Your own message here...");
    else
        $display("Test vector passed!!!");
//okay this is fun... you try it now...

//go through all possible input combinations (2^4 = 16)
//cut and paste makes this task a lot easier

//when we are done, let's stop the simulation
{A,B} = 4'b1000; //stimulate the inputs
#25; //wait a bit for the result to propagate
//check output
if({Carry, Sum} != 3'b010)
    $display("Your own message here...");
    else
        $display("Test vector passed!!!");
//okay this is fun... you try it now...

//go through all possible input combinations (2^4 = 16)
//cut and paste makes this task a lot easier

//when we are done, let's stop the simulation
{A,B} = 4'b1001; //stimulate the inputs
#25; //wait a bit for the result to propagate
//check output
if({Carry, Sum} != 3'b011)
    $display("Your own message here...");
    else
        $display("Test vector passed!!!");
//okay this is fun... you try it now...

//go through all possible input combinations (2^4 = 16)
//cut and paste makes this task a lot easier

//when we are done, let's stop the simulation

{A,B} = 4'b1010; //stimulate the inputs
#25; //wait a bit for the result to propagate
//check output
if({Carry, Sum} != 3'b100)
```

```
{A,B} = 4'b1010; //stimulate the inputs
#25; //wait a bit for the result to propagate
//check output
if({Carry, Sum} != 3'b100)
    $display("Your own message here...");
    else
        $display("Test vector passed!!!");
//okay this is fun... you try it now...

//go through all possible input combinations (2^4 = 16)
//cut and paste makes this task a lot easier

//when we are done, let's stop the simulation

{A,B} = 4'b1011; //stimulate the inputs
#25; //wait a bit for the result to propagate
//check output
if({Carry, Sum} != 3'b101)
    $display("Your own message here...");
    else
        $display("Test vector passed!!!");
//okay this is fun... you try it now...

//go through all possible input combinations (2^4 = 16)
//cut and paste makes this task a lot easier

//when we are done, let's stop the simulation
{A,B} = 4'b1100; //stimulate the inputs
#25; //wait a bit for the result to propagate
//check output
if({Carry, Sum} != 3'b011)
    $display("Your own message here...");
    else
        $display("Test vector passed!!!");
//okay this is fun... you try it now...

//go through all possible input combinations (2^4 = 16)
//cut and paste makes this task a lot easier

//when we are done, let's stop the simulation
{A,B} = 4'b1101; //stimulate the inputs
#25; //wait a bit for the result to propagate
//check output
if({Carry, Sum} != 3'b100)
    $display("Your own message here...");
    else
        $display("Test vector passed!!!");
//okay this is fun... you try it now...

//go through all possible input combinations (2^4 = 16)
//cut and paste makes this task a lot easier

//when we are done, let's stop the simulation
```

```
        {A,B} = 4'b1110; //stimulate the inputs
        #25; //wait a bit for the result to propagate
        //check output
        if({Carry, Sum} != 3'b101)
            $display("Your own message here...");
            else
                $display("Test vector passed!!!");
        //okay this is fun... you try it now...

        //go through all possible input combinations (2^4 = 16)
        //cut and paste makes this task a lot easier

        //when we are done, let's stop the simulation
        {A,B} = 4'b1111; //stimulate the inputs
        #25; //wait a bit for the result to propagate
        //check output
        if({Carry, Sum} != 3'b110)
            $display("Your own message here...");
            else
                $display("Test vector passed!!!");
        //okay this is fun... you try it now...

        //go through all possible input combinations (2^4 = 16)
        //cut and paste makes this task a lot easier

        //when we are done, let's stop the simulation



        $stop;
    end

endmodule
```

Figure 8. Adder Synchronous

```
`timescale 1ns / 1ps

module adder_synchronous(Carry_reg, Sum_reg, Clk, A, B);

    //output ports are reg
    output reg Carry_reg;
    output reg [1:0] Sum_reg;

    //inputs are still wires
    input wire Clk;
    input wire [1:0] A, B;

    //inteermediate nets
    reg [1:0] A_reg, B_reg;
    wire Carry;
    wire [1:0] Sum;

    //instantiate 2 bit adder here
    adder_2bit a2b0(Carry, Sum, A_reg, B_reg);

    //this is a beahvioral block desceibes two 2-bit registers
    //remembers registers are nothing more than grouped flipflops
    always@(posedge Clk)
        begin
            A_reg <= A;
            B_reg <= B;
        end
    always@(posedge Clk)
        begin
            Carry_reg <= Carry;
            Sum_reg <= Sum;
        end

endmodule
```

Figure 9. 2ns

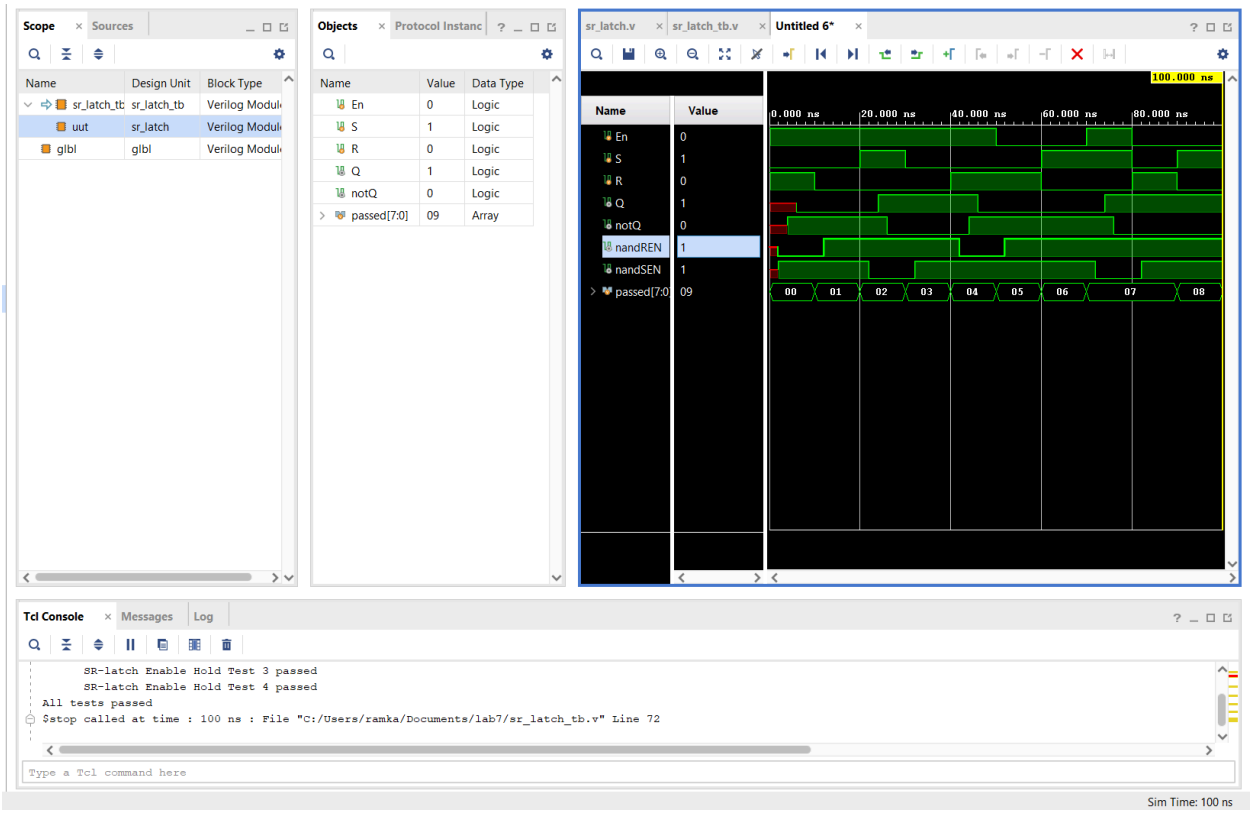


Figure 10. 4ns

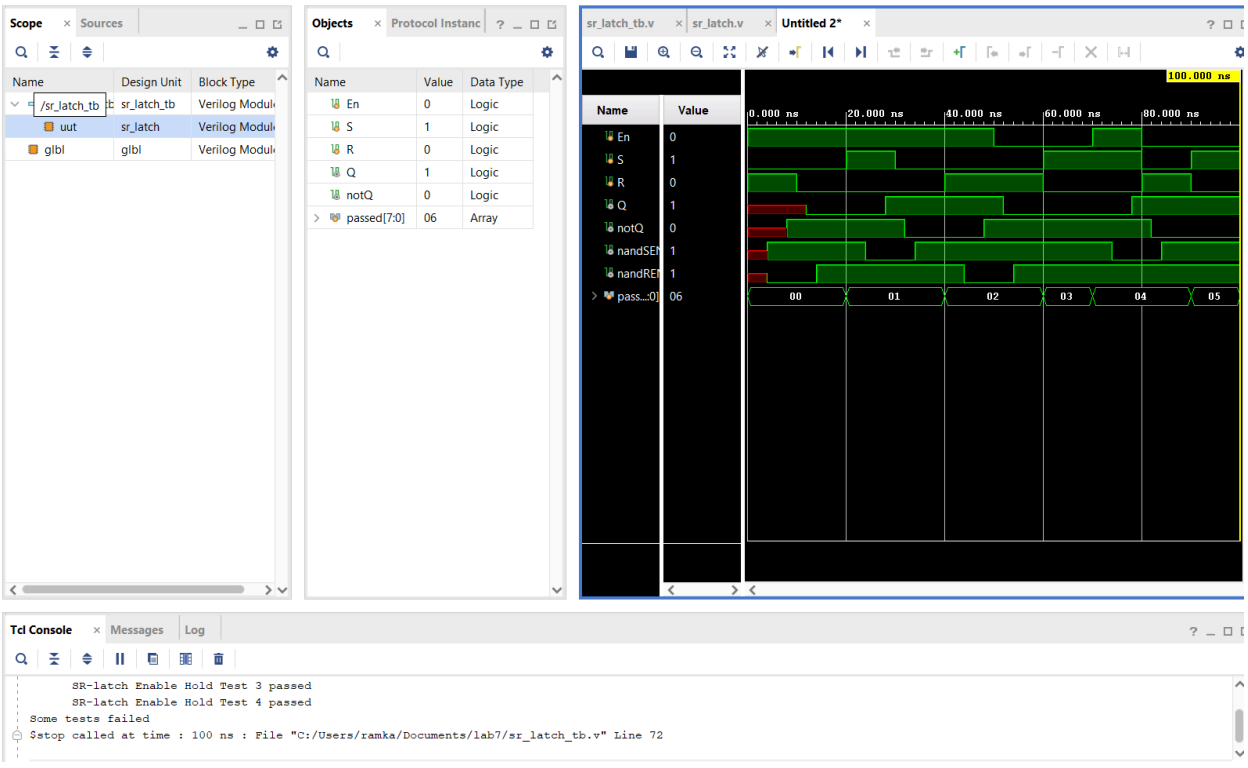


Figure 11. D-latch Structural

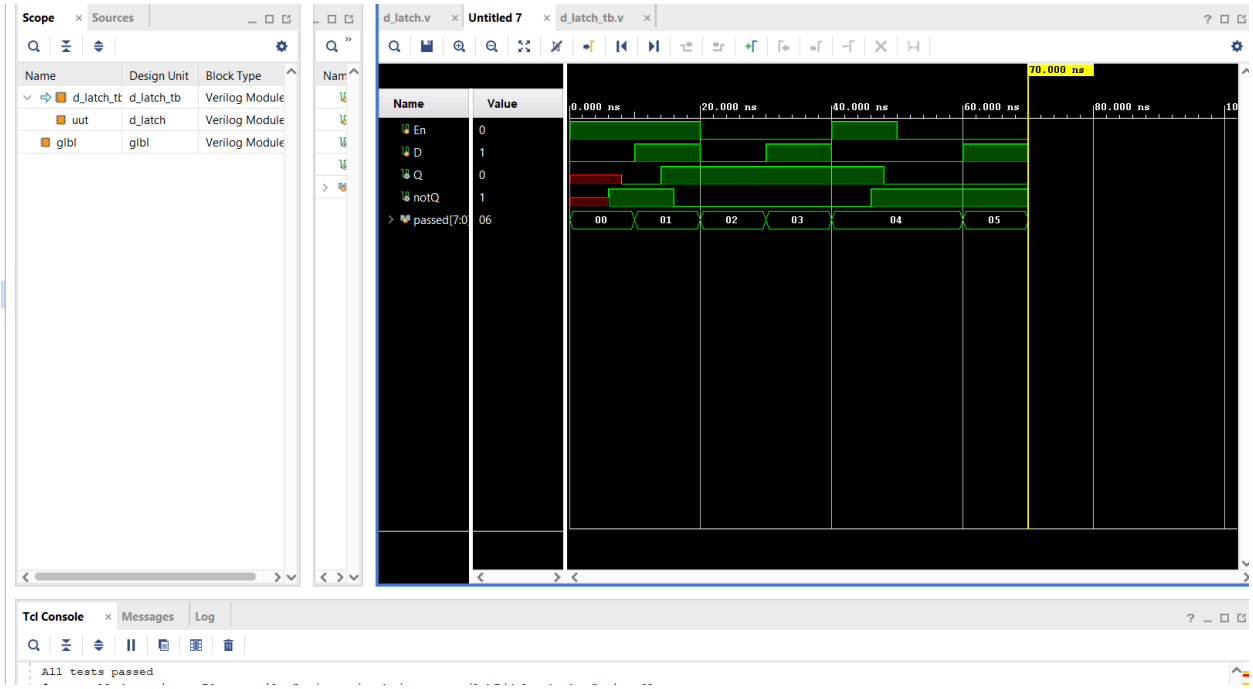


Figure 12. D-latch Behavioral

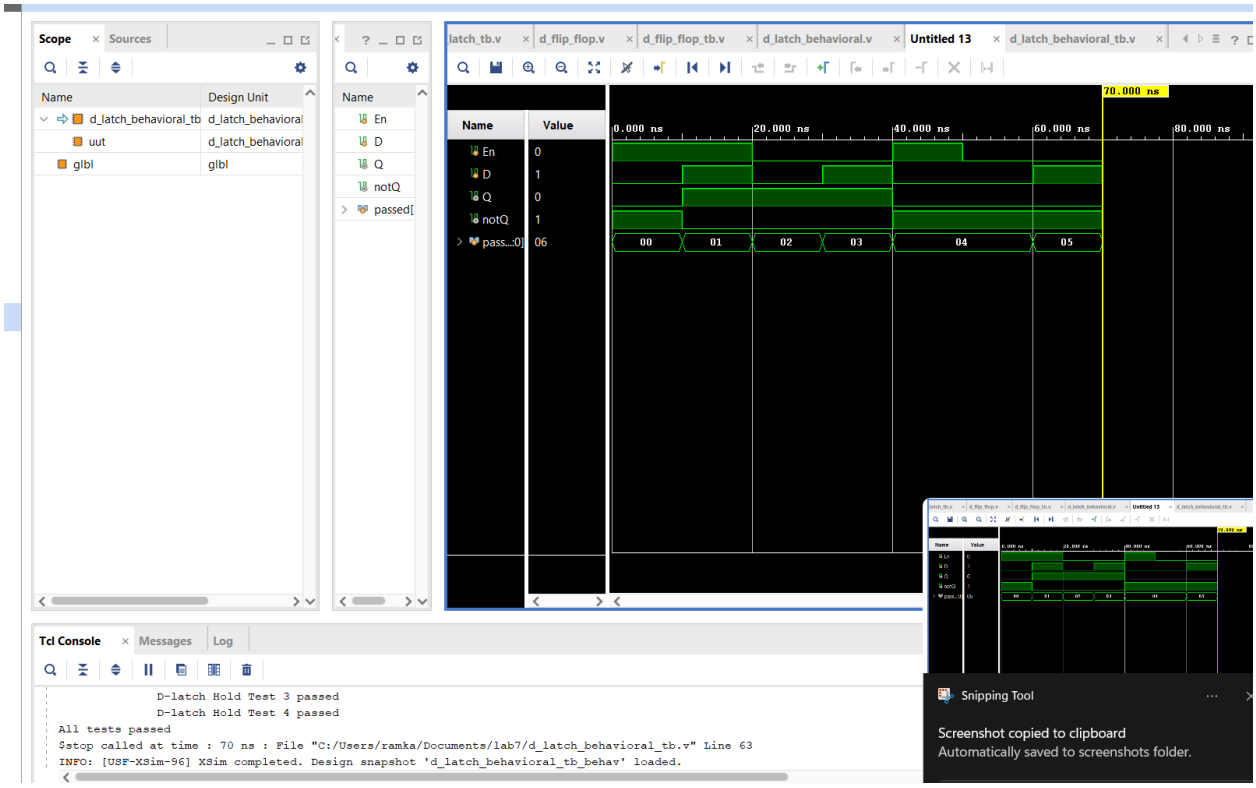


Figure 13. D Flip-Flop Structural

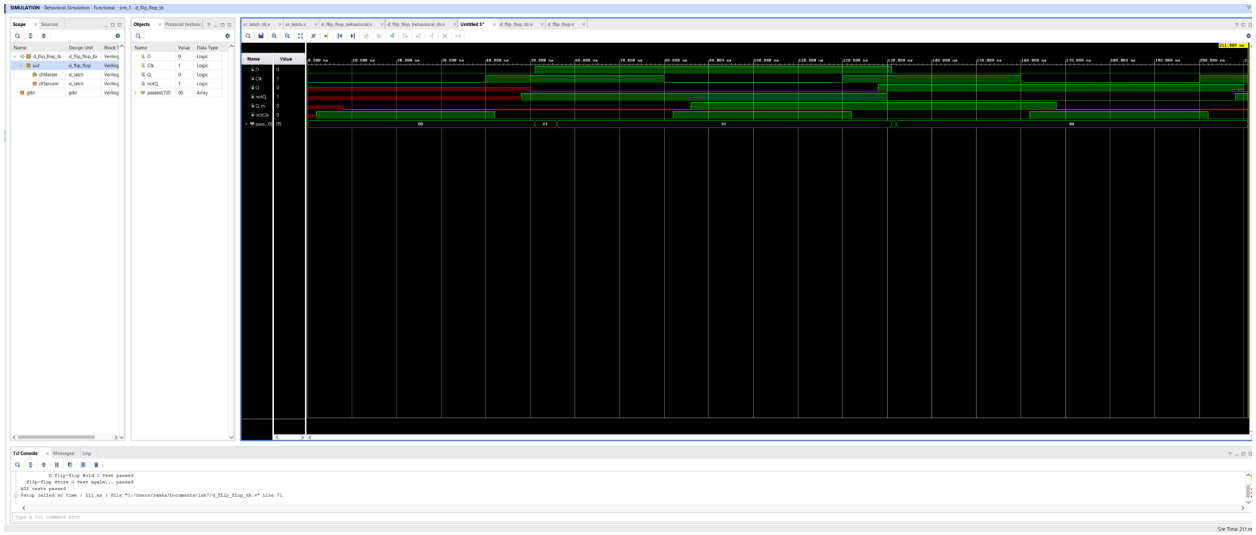


Figure 14. D Flip-Flop Behavioral

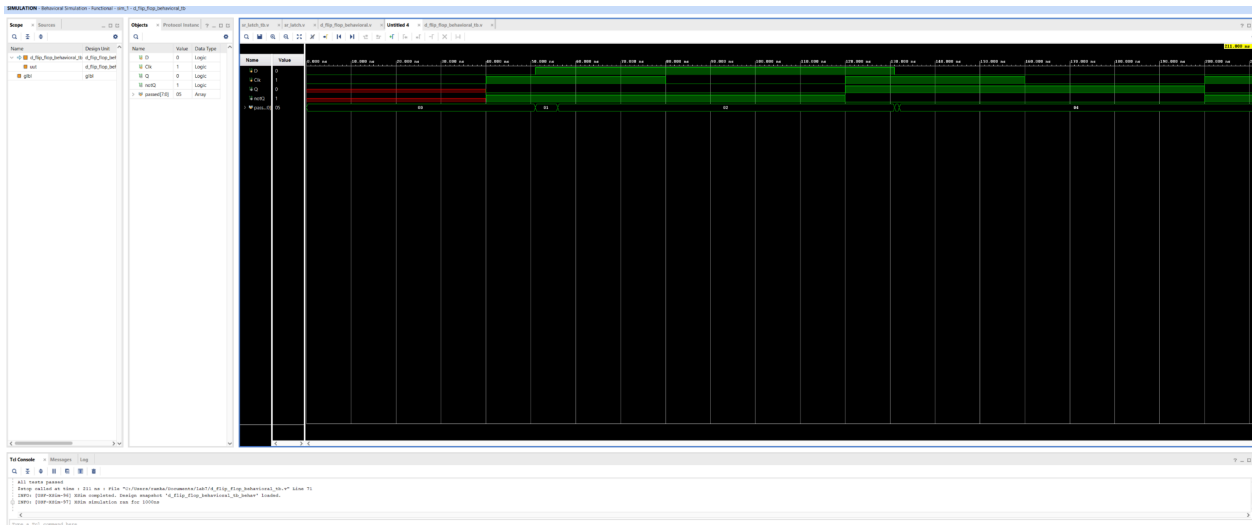
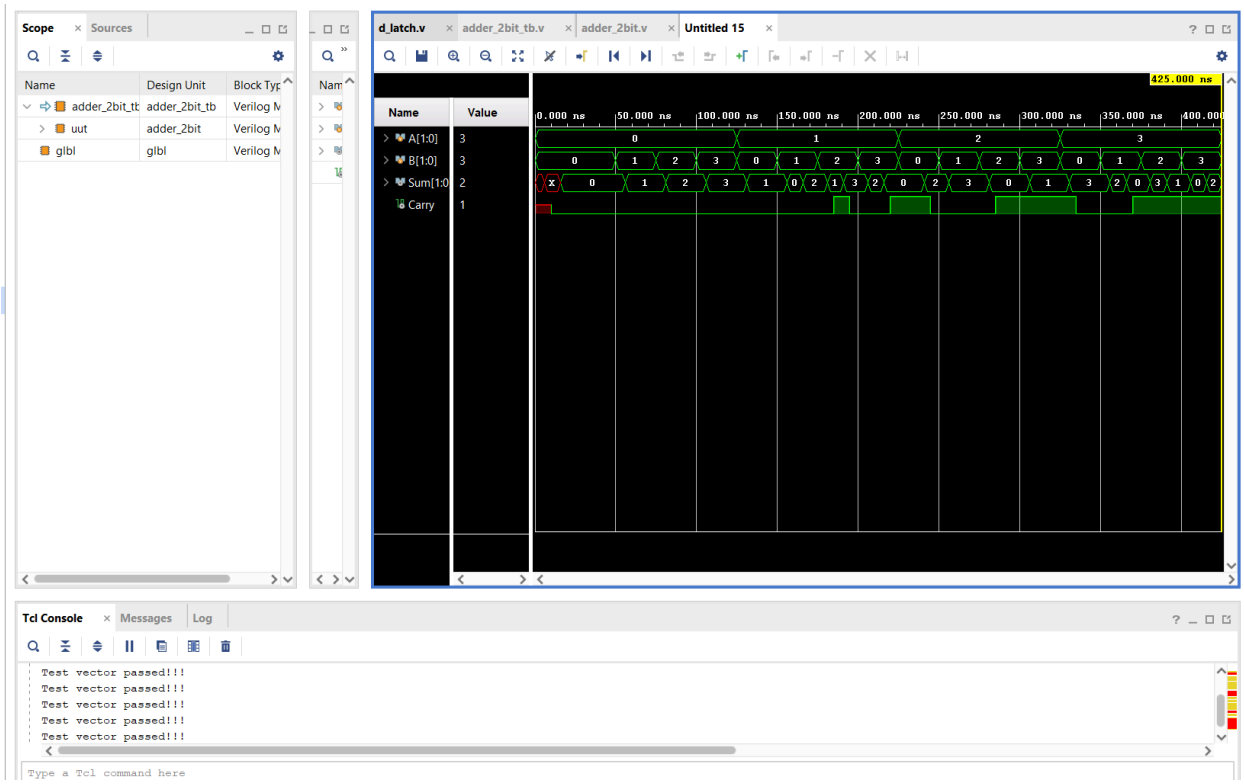


Figure 15. 2-bit Adder




```
# run 1000ns
Hey! The UUT passed this test vector...
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
Test vector passed!!!
```

Figure 16. 18 ns Clock Period (Some tests failed)

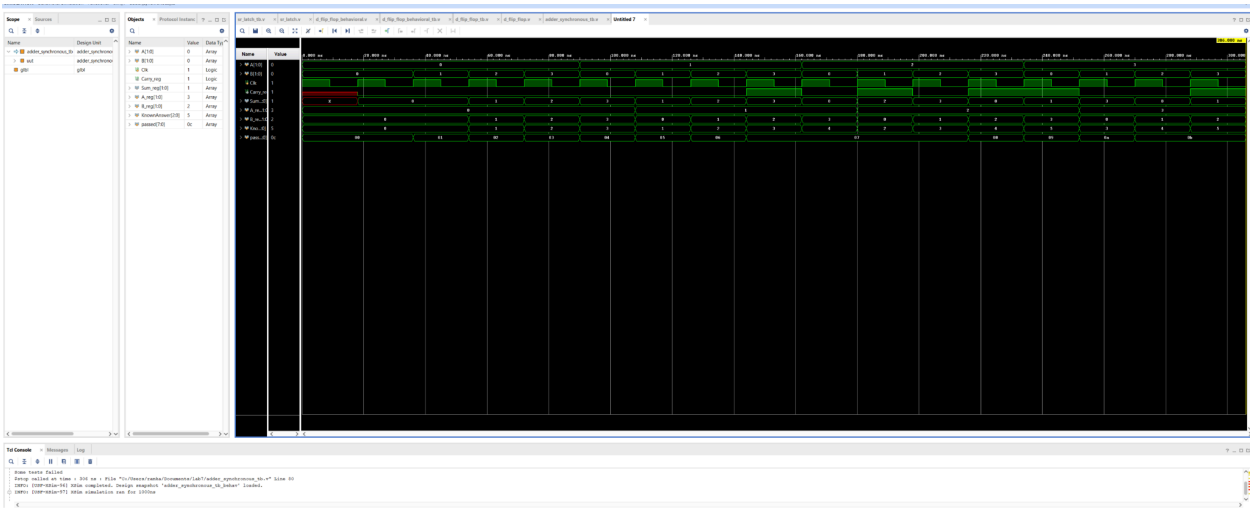


Figure 17. 19 ns Clock Period

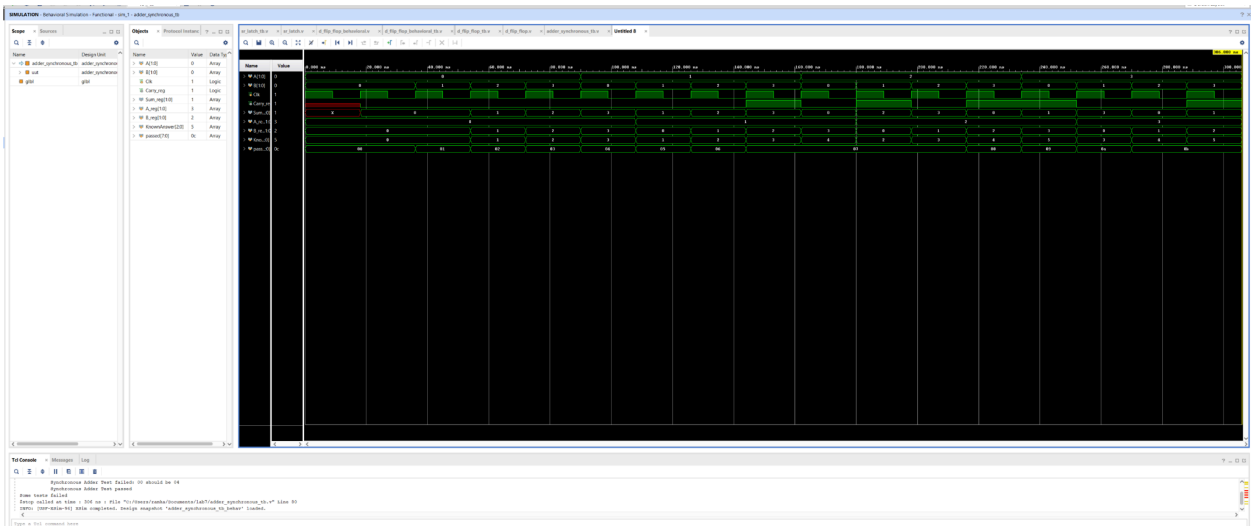
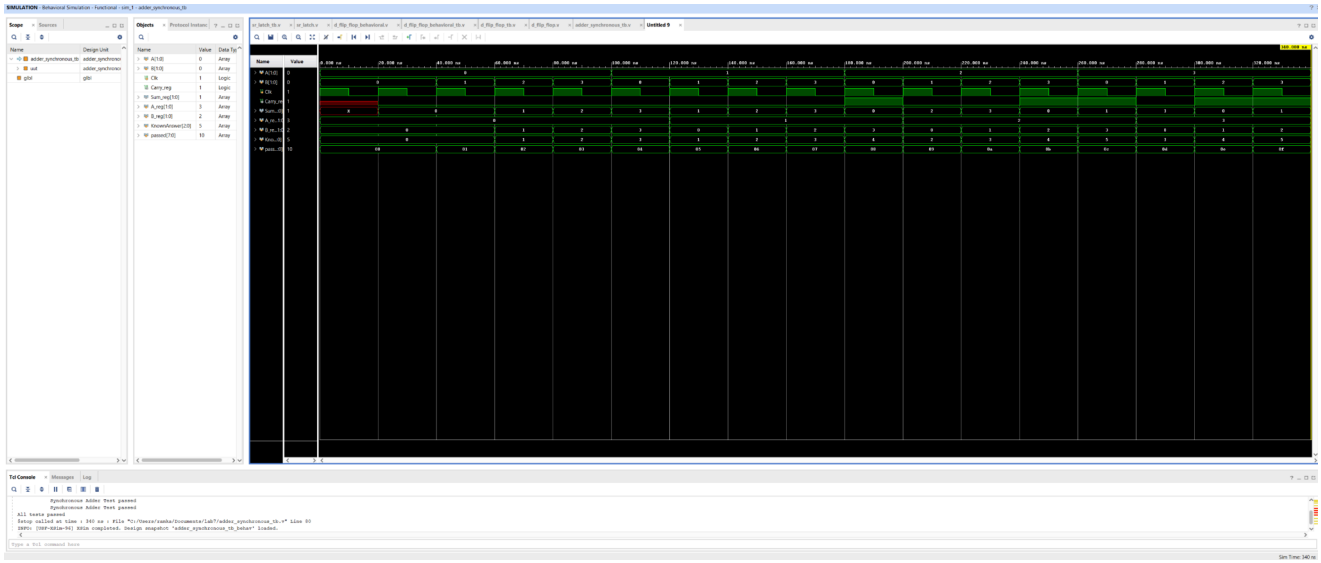


Figure 18. 20 ns Clock Period (All tests passed)


SIMULATION
Scope
Objects
Untitled 9
Tcl Console
Messages
Log
All tests passed
Type a Tcl command here